DTU Compute
Department of Applied Mathematics and Computer Science

Towards Intermittent Neuromorphic and Embedded Computing for Battery-Free Wearable Medical Devices

Master Thesis



Towards Intermittent Neuromorphic and Embedded Computing for Battery-Free Wearable Medical Devices

Master Thesis
August, 2024

By
Junaid Qazi

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Brovej, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk

Approval

This thesis was prepared over six months at the Embedded Systems Engineering research group at the Department of Applied Mathematics and Computer Science, DTU, in partial fulfilment of the degree Master of Science in Engineering, M.Sc Eng.

Junaid Qazi - s213267

.....

Signature

.....

Date

Abstract

Wearable medical devices have become a focal point of research and development, particularly for the long-term monitoring of patients with chronic illnesses. These devices offer significant advantages, including the early detection of complications, reduced hospital readmission rates, and overall lower healthcare costs by enabling a more proactive approach to patient care. In recent years, the integration of artificial intelligence (AI) into wearable technology has gained considerable attention, further enhancing the capabilities of these devices. However, a notable challenge associated with AI-driven wearables is the increased power consumption, which can limit their practicality for continuous use.

This thesis presents a novel approach that combines neuromorphic computing with a particular focus on Spiking Neural Network (SNN) and intermittent computing. This method is applied to an Electromyography (EMG) dataset for the classification of various hand gestures.

Our findings demonstrate that intermittent computing, specifically through the use of checkpointing, can be effectively integrated within SNNs. Moreover, we observe that the overhead, frequency, and optimal placement of checkpointing are highly dependent on the architecture of the SNN model and the associated inference overhead. While the introduction of checkpointing in the processing of our EMG dataset results in a relatively low energy overhead, it leads to a significant latency overhead.

Acknowledgements

I would like to express my appreciation towards everyone who assisted me through this project. Specifically, Emil, Haris, Jens and Fontas, for their great insights and supervision. Lastly, I would like to thank Maratuneski for the support and ideation.

Contents

Preface ii

1 Introduction **1**

2 Background **2**
2.1 Electromyography 2
2.2 Neural Networks 3
2.3 Neuromorphic Computing 4
2.4 Spiking Neural Networks 4

3 Related Works **7**
3.1 EMG Deep Learning 7
3.2 EMG SNN 8
3.3 Intermittent Computing 8

4 Feasibility Study **10**
4.1 Phase 1 - ImC SNN Dynamics 10
4.2 Phase 2 - ImC SNN vs SNN Trade-offs 16
4.3 Phase 3 - ImC SNN Strategy 20

5 SNN-EMG Case Study **21**
5.1 SNN Model 21
5.2 Intermittent EMG SNN 25

6 Discussion **33**
6.1 General Intermittent Computing (ImC) SNN 33
6.2 ImC SNN EMG 33
6.3 Future Works 34

7 Conclusion **35**

References **36**

A Inference Energy Calculations **39**

Acronyms

AI Artificial Intelligence.

ANN Artificial Neural Network.

CSNN Convolutional Spiking Neural Network.

EEG Electroencephalogram.

EMG Electromyography.

FRAM Ferroelectric Randam Access Memory.

ImC Intermittent Computing.

IMU Inertial Measurement Unit.

Mem Membrane Potential.

NVMe Non-Volatile Memory.

SNN Spiking Neural Network.

SOP Synaptic Operation.

Spk Spiketrain.

Syn Synaptic Current.

ts Timestep.

Chapter 1

Introduction

There has been a large ongoing effort in researching and developing wearable medical devices for long-term monitoring of chronically ill patients. There are many advantages to long-term monitoring using wearables, such as early detection of complications, reduced hospital readmission (and thus lower overall healthcare costs), and a more proactive approach to healthcare. In recent years, there has been a major increase in research on using artificial intelligence in wearable medical devices. A major advantage of AI in medical wearables is its ability to predict certain complications (e.g. strokes, arrhythmia). However, its increased power consumption is a major disadvantage of integrating AI in wearables.

Different solutions to this problem have been proposed over the years. However, there is no general consensus yet on the best approach for medical AI wearables. This thesis proposes a potential solution that combines neuromorphic computing, with a specific focus on SNN, and intermittent computing. This approach is applied to an Electromyography dataset, where different gestures are classified.

The event-based nature of SNNs makes them multiple orders of magnitude more energy efficient than conventional AI architectures. On the other hand, Intermittent Computing allows devices to run on non-continuous power sources (e.g., energy harvesters). Combining SNNs with an Intermittent Computing algorithm could result in a wearable that can run off energy harvesting while still having the same AI capabilities that conventional, less energy-efficient AI architectures have.

Objectives and Scope
The main objectives of the project are:

- Assessing the possibility of making an SNN intermittent
- Assessing the trade-offs between a non-intermittent and intermittent SNN
- Evaluating the effects of the intermittent SNN on the Electromyography dataset

Due to the project's exploratory nature, the scope will be mostly limited to the theoretical aspects of the proposed concept. This entails creating algorithms and then evaluating the performance quantitatively.

Chapter 2

Background

2.1 Electromyography

EMG is a technique for measuring electrical activity generated by voluntary or involuntary muscular activation. At rest, a muscle fibre maintains a resting membrane potential at -80 mV, resulting from different concentrations of ions between the intracellular and the extracellular space. The muscle fibre is activated when a voltage threshold is crossed. The threshold is reached after action potentials from the spinal cord cause a positive influx of ions to the intracellular space and make the membrane potential more positive. This depolarization propagates along the muscle fibre and is followed by subsequent repolarization.

EMG signals are measured by electrodes placed on the skin over the muscles (surface EMG) or by inserting a needle electrode into the muscle (intramuscular EMG). The signal obtained represents the voltage generated in this muscular activation between at least two electrodes. Commonly, one of the electrodes acts as a reference and is placed in a neutral position, such as a tendon or bone. The voltage obtained from an electrode placed on top of the muscle fibre groups results from the summation of the individual motor unit (neuron cell and innervations) action potentials, which can vary depending on the fibre orientation relative to the electrode.

The amplitude of the action potential increases with the magnitude of muscular activation. Figure 2.1 illustrates two EMG signals at a low (10%) and high (40%) maximum voluntary contraction (MVC) of a muscle group. At the low MVC, action potentials are more easily distinguishable than at higher MVC levels due to the increased motor unit action potentials that are summed at more time steps [1] [2].

The recorded signals are amplified, filtered, and sampled, after which they either become part of a control loop or are stored for later analysis.

EMG signals can provide information about local motor activation and be used to assess, for example, the count of motor units activated or activation/relaxation timings. In this way, EMG can be used as a diagnostic tool for medical conditions that affect the muscles and nerves, such as Parkinson's Disease and Multiple Sclerosis.

In recent years, efforts have been made to use artificial intelligence to classify EMG signals, commonly using temporal or spectral features.

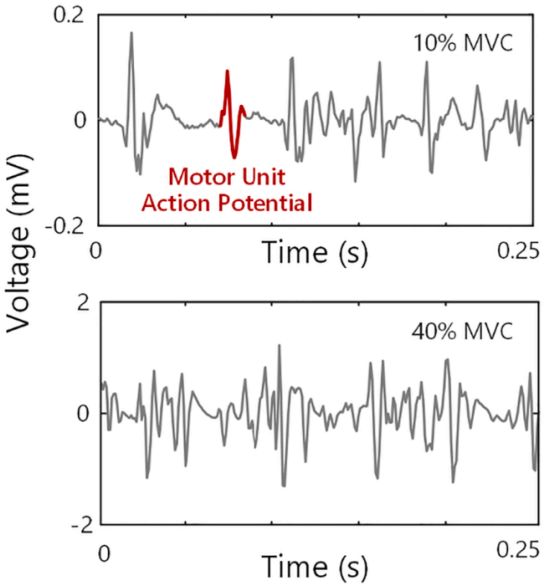


Figure 2.1: Surface EMG signals at a low force level (10% of MVC) and a higher force level (40% MVC) in the first dorsal interosseous muscle (Tr = 0.25 s). Adopted from [1].

2.2 Neural Networks

Neural networks (NN) are systems of interconnected neurons that transmit information, enabling learning processes. This concept has been digitally adapted to create 'artificial' neural networks, which can similarly learn from a given input set of data. The most basic neural network is given by three layers, the input layer, the hidden layer, and the output layer. The input data is connected to the hidden layer, that can be composed of just one neuron unit, to which is given the input data that is weighted and biased. In the neural unit, these data undergo an activation that allows for decision-making and pattern detection and is commonly a function such as a sigmoid or a rectified linear unit [3].

Several types of neural networks have emerged since they were introduced by Rosenblatt in 1958 [4]. The networks can differ in several aspects, such as the type of learning, the layer architecture type, and the type of activation.

Feedforward networks (FNN) are the archetype of NN. In the forward pass, data flows from input to output, where a prediction is made. The success of this prediction can inform other forward passes. The network learns via backpropagation, which is a method that works backward from the output layer to the input layer, calculating the gradient of the loss function with respect to each weight and bias in the network. The weights and biases are updated iteratively in each backward pass by minimizing the loss function [5].

Deep neural networks (DNN) are an umbrella term that includes model architectures with multiple hidden layers. A deep structure enables complex learning and comes with a computational and overfitting cost.

Recurrent neural networks (RNN) are a type of DNN and a variation of FNN, in which the forward pass is composed of multiple steps, instead of just one. The output of a certain current cell informs the input of the next and since the cells have built-in memory this architecture allows for learning in sequence.

Convolutional neural networks (CNN) convolve kernels of input data, making them appropriate for identification of spacial patterns.

Generative Adversarial Networks (GAN) consist of two competing neural networks: a generator and a discriminator. The generator's task is to create data samples, while the discriminator's role is to predict whether these samples are real (from the training data) or generated. The discriminator's success in making accurate predictions provides an error gradient that guides the generator in improving its outputs. GANs are used in applications such as image synthesis and data augmentation.

The autoencoders learn by compressing (encode) and regenerating (decode) the data. The autoencoder first compresses the input data into a lower-dimensional representation, a bottleneck output. The network then attempts to reconstruct this output through a decoder. The loss function measures how closely the reconstructed output matches the original input. Autoencoders are used in dimensionality reduction, data denoising and feature learning.

Both Hopfield networks and Boltzmann machines are associative memory models and work by clamping desired patterns in input neurons, representing a pattern the network should learn. During training, the network adjusts its weights and stores this pattern for future recall. In Hopfield networks, the model adjusts the weights so that the pattern clamped becomes a stable state. In Boltzmann machines, the weights are adjusted to model the probability distribution of the input data [5].

In spiking neural networks (SNN), the activation of a neural unit occurs when a threshold is reached, much like what is seen when a membrane potential reaches a threshold and activates the biological neuron. Further details on this NN are provided in future sections.

2.3 Neuromorphic Computing

Neuromorphic computing is a paradigm inspired by the structure of the brain, closely related to the field of brain-inspired computing research. Unlike traditional computing systems, neuromorphic computers do not rely on the Von Neumann or Harvard architectures. Instead, they use neurons and synapses for memory and processing, mimicking the behaviour of biological neurons. This approach offers advantages such as greater energy efficiency and enhanced parallelism. Active research in neuromorphic computing spans both hardware design and algorithm development [6].

Among the algorithms developed for neuromorphic computing are SNNs, echo state networks, liquid state machines, and synaptic plasticity models. This project focuses on SNNs, as they are more commonly used in the context of medical devices, which is the primary application area of this research. From this point on, neuromorphic computing and SNN will be used interchangeably.

2.4 Spiking Neural Networks

Spiking Neural Networks (SNN) are neural networks that more closely mimic the behaviour of the brain compared to traditional neural networks. As the name suggests, SNNs use spikes instead of continuous numerical values. These spikes are discrete events, and the timing of these spikes is incredibly important for the SNN. This is another difference between SNNs and traditional neural networks, as SNNs leverage the temporal dynamics of the input spikes. In a more practical sense, an SNN consists of three main parts:

- Input Encoding
- Neuron Model
- Output Decoding

Figure 2.2 shows a general overview of an SNN with input encoding and output decoding

2.4.1 Input Encoding

Oftentimes, the dataset used by an SNN is the same dataset that is used for traditional neural networks. To be able to fully use the temporal dynamics of the SNN, the dataset needs to be converted into spikes. This is done using input encoding techniques, which convert continuous numerical values into spikes. The three most common input encoding techniques are:

- Rate Encoding: Converts the magnitude of a continuous input signal into the frequency of spikes generated within a specified time window
- Latency Encoding: Converts the magnitude of a continuous input signal into a timing delay of the first spike generated by a neuron - higher magnitudes result in a neuron firing its spike quickly, while lower magnitudes have the opposite results
- Delta Encoding: Represents changes in input signals, and thus only fires a spike when the difference between two input signals is above a certain threshold

2.4.2 Neuron Model

The neuron model in an SNN determines how neurons process incoming spikes and when they decide to fire a spike. This simulates the electrical behaviour of a biological neuron by integrating incoming spikes over time and firing when a certain threshold is reached.

Neuron models in an SNN replace the activation function of a traditional neural network, which is another major difference between these two networks. There are different types of neuron models, such as the Leaky-Integrate and Fire neuron model (LIF), which represents a simple RC circuit. The LIF neuron model has the following hidden states:

- Membrane Potential: Represents the electrical state of the neuron and determines whether the neuron will fire a spike. If a neuron receives a spike from a presynaptic neuron, it will increase its membrane potential.
- Threshold: Determines when a neuron membrane potential is high enough to fire a spike
- Voltage Decay: After a neuron receives a spike from a presynaptic neuron, and thus its membrane potential has been increased, it will decay at a certain rate.
- Spike Train: Spikes that are created by a neuron reaching its membrane potential

A Synaptic-LIF model is an enhanced version of the standard LIF model, incorporating greater biological realism. In addition to the membrane potential dynamics present in the LIF model, the Synaptic-LIF also accounts for synaptic currents. Unlike the LIF model, where the membrane potential experiences an immediate jump upon receiving a spike, the Synaptic-LIF model introduces synaptic currents that cause the membrane potential to increase more gradually. This gradual increase better mimics the temporal dynamics of real synaptic transmission.

2.4.3 Output Decoding

While the input data of an SNN had to be converted into spikes, the output of an SNN needs to do the exact opposite. While traditional neural networks select the class with the highest probability for its classification, neural networks use the dynamics of the output spikes of its final layer as a classification. The most common methods for output decoding are:

- Rate Decoding: Measure the number of spikes produced by a neuron over a period of time, the neuron with the largest number of spikes is the predicted class
- Temporal Decoding: Use the exact timing of when spikes were received to decode classification, the neuron that spikes first indicates the predicted class
- Latency Decoding: Use the latency between multiple spikes to determine its class

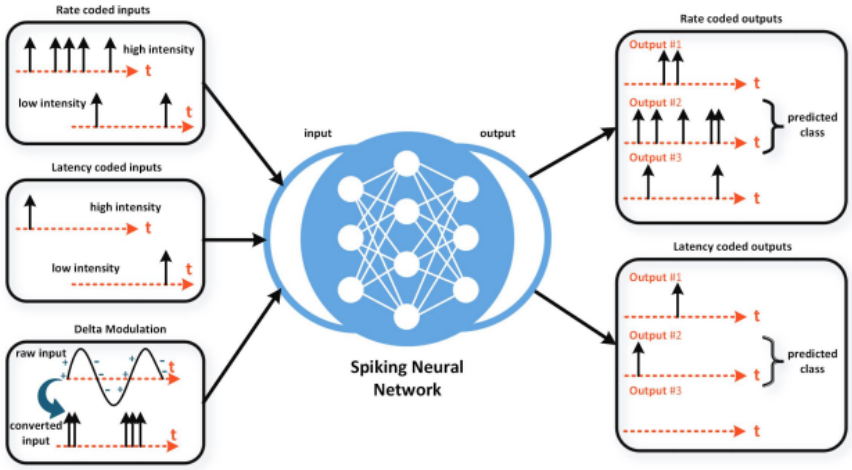


Figure 2.2: General overview of input encoding and output decoding in an SNN [7]

Chapter 3

Related Works

3.1 EMG Deep Learning

EMG signals have been used with deep learning techniques to perform several tasks, including classifications on hand gestures, classification of postures and angles of other body parts, force estimation, disease diagnoses, fall detection, and, combined with other data types, sign language recognition.

Focusing on EMG signals and hand gesture classification, the input data to the classifier is typically the time-domain data of superficial EMG signals.

Electrodes are typically placed on the forearm, or on the hand with a glove. The latter is commonly applied with the Cyberglove that is used in the publicly available NinaPro databases, which are used in several records [8].

The Ninapro DB2 is one of the most common public datasets used and contains 50 hand gestures from 40 subjects. Using this dataset, Ding et al. [9] achieved a maximum average recognition accuracy of 78.86%, using the EMG time series as input to a CNN with two parallel blocks. With the same dataset, Hu et al. [10] created an attention-based hybrid CNN-RNN architecture that is fed with a time and frequency domain feature set that achieves 82.2% of classification accuracy. Under the same context, Atzori et al. [11] obtained an average accuracy of 60.27% with an architecture constituted by 4 CNN layers which was fed with time-domain EMG data.

The power consumption of a deep learning model while in training and deployment is relevant for applications such as wearables, portable medical devices and prosthetics. Tam et al. [12] developed an embedded online convolutional neural network to classify hand-muscle contractions sensed at the forearm with 32 surface channels. Only one participant was recorded and the EMG signals were preprocessed with a frequency-time-space cross-domain method. The embedded CNN classifies 8 gestures with an accuracy average of 98.15%. The sensor subsystem (including the wireless communication) consumes 49.5 mW. During continuous inference, the microcontroller NVidia Jetson Nano that received the signals consumes 3.055 W.

Zanghieri et al. [13] also proposed a real-time embedded EMG classification method. The architecture was composed of three blocks (each with 2 convolutional and one pooling layer), followed by two fully connected layers. Two datasets were used, the Ninapro DB6 (14 channels, 8 classes) with which an average accuracy of 49.6% was achieved, and a self-acquired dataset (8 channels, 8 classes, 20 sessions) obtaining an accuracy of 93.7%. Considering the classification energy, each 15 ms time window cost 0.90 mJ per inference

on a multicore low-power GAP8 processor, which yielded an average power consumption of 60 mW.

3.2 EMG SNN

Several studies have explored the implementation of SNNs for EMG applications.

Vitale et al. [14] developed an SNN for gesture classification using EMG signals from the Ninapro DB5 dataset. They evaluated two types of models: fully connected SNNs and convolutional SNNs. Multiple architectures were tested for each model type, with the best-performing convolutional SNN achieving a test accuracy of 70%, and the best fully connected SNN achieving a test accuracy of 74%. Additionally, Vitale et al. [14] implemented their model on an Intel Loihi neuromorphic chip, finding that the most successful model had a total power consumption of 41 mW and a latency of 5.7 ms.

Bezugam et al. [15] developed an SNN for gesture classification using EMG signals from the Roshambo EMG dataset. They designed a recurrent SNN that achieved test accuracy ranging from 84.7% to 90%. Similar to the work of Vitale et al. [14], this SNN was also implemented on the Intel Loihi neuromorphic chip, with an energy consumption rate of 0.37 mJ per inference. Although no specific latency value was provided, it was noted that the latency was reportedly lower than that of CPU and GPU implementations for EMG datasets.

Ceolini et al. [16] developed an SNN for gesture classification using a self-acquired dataset. For this dataset, they utilized five surface EMG electrodes and an event-based camera. The SNN achieved a test accuracy of 96%. Implemented on an Intel Loihi neuromorphic chip, the model exhibited a total energy consumption of 1104 uJ and a latency of 7.75 ms.

3.3 Intermittent Computing

Intermittent computing is an emerging paradigm that operates on non-continuous energy resources, such as those provided by energy harvesting. This approach is particularly relevant for devices requiring low power, including those used in the Internet of Things and wearables. As the demand for such devices grows, research within intermittent computing has expanded across multiple domains, including the development of specialized hardware, algorithms, and programming languages [17] [18]. This field continues to evolve, addressing energy-constrained environments' unique challenges and opportunities.

3.3.1 Checkpointing

In the context of intermittent computing, checkpointing refers to the process of saving a device's state so that it can resume from that point after a power loss. Traditionally, devices are designed to operate on a continuous energy source. However, with the advent of intermittent computing and energy harvesting, devices often face interruptions in power supply, leading them to revert to their initial state after a failure. Checkpointing addresses this challenge by enabling the device to return to an intermediate state instead of restarting from the beginning, thereby improving efficiency and reducing redundancy [19].

Jayakumar et al. [20] proposed a checkpointing method that employs Ferroelectric RAM (FRAM) as a non-volatile memory (NVM) solution, known for its lower latency and higher energy efficiency compared to other NVM technologies. They evaluated this method using 16- and 32-bit cyclic redundancy checks, achieving a total latency overhead of just 12.6 ms. This result marked a significant advancement at the time, with performance that was orders of magnitude faster than traditional NVM solutions.

Maeng et al. [21] introduced "Chinchilla," a compiler designed to insert checkpoints into standard C code automatically. Chinchilla employed an adaptive checkpointing mechanism, adjusting the checkpointing frequency based on varying energy conditions. This approach resulted in Chinchilla being 125% faster than other checkpointing compilers.

Colin et al. [22] developed a tool designed to detect and prevent non-terminating paths in task-based intermittent programs. Non-terminating paths are defined as sequences within a task that consume more energy than the device's energy storage can supply before depletion (e.g. when a supercapacitor drops below its threshold voltage). The tool proved to be faster and more efficient than manually identifying these paths.

3.3.2 Intermittent Learning

Intermittent learning is an up-and-coming field within ImC that explores intermittent machine- and deep-learning models. In this case, the general principles of checkpointing are applied to these models so that a previous state can be returned after power loss instead of restarting during inference. In terms of research areas, intermittent learning is the closest area to our project.

Lee et al. [23] proposed an intermittent computing scheme tailored for semi-supervised and unsupervised machine learning models. They distinguished between two application types of intermittent learning: real-time and semi-real-time. In real-time systems, sensor data is sporadic, whereas in semi-real-time systems, sensor data is relatively constant, allowing the intermittent learning algorithm to operate in a best-effort manner. Lee et al. [23] implemented their proposed scheme in three applications: solar-harvesting-based air quality monitoring, RF-harvesting-based human presence detection, and piezoelectric-harvesting-based vibration detection. The results showed an increase in energy efficiency of up to 100% compared to existing methods. Notably, the study also provided valuable insights into the dynamics of semi-supervised and unsupervised learning within intermittent computing.

Islam et al. [24] introduced Zygarde, an imprecise computing-based scheduling algorithm for neural networks. This algorithm divides deep learning tasks into mandatory and optional components, enabling the system to prioritize and execute critical parts of tasks when energy is available, thereby ensuring task deadlines are met. Additionally, the algorithm models the energy harvesting pattern to predict available energy, which is then used to optimize task scheduling. The results demonstrated that Zygarde reduced execution time by up to 26% compared to other scheduling algorithms.

Gobieski et al. [25] introduced "SONIC," a software system designed to manage deep learning models on intermittent systems. The approach began with significant compression of the deep learning models to reduce their memory footprint, making them suitable for low-power microcontrollers. Subsequently, SONIC introduced a method for dynamically resizing tasks based on the available energy, ensuring that computation could continue even in the face of intermittent power. Unlike traditional checkpointing, SONIC tracks the completion of individual loop iterations, allowing the system to resume from the exact point of interruption after a power failure, thereby minimizing redundant work and ensuring error-free re-execution. In terms of performance, SONIC demonstrated a twofold improvement in execution time compared to other frameworks, such as Alpaca. Additionally, SONIC's dynamic task resizing minimized stalling and ensured consistent forward progress.

Chapter 4

Feasibility Study

To the best of our knowledge, no previous studies have explored the implementation of ImC within SNNs. This gap highlights the need for a feasibility study to examine the interaction between SNNs and ImC. The scope of this feasibility study can be divided into three phases:

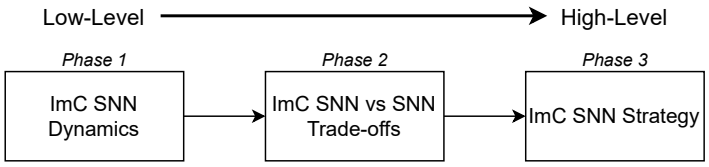


Figure 4.1: Overview of the research scope within intermittent computing

These phases examine ImC and SNNs across different levels of abstraction. In Phase 1, we investigate the overall dynamics of an SNN and identify critical elements relevant to ImC. The primary goal of this phase is to assess the feasibility of making an SNN intermittent. If the results from Phase 1 are favourable, we proceed to Phase 2, where we compare an intermittent SNN with a non-intermittent SNN. This phase focuses on identifying the trade-offs in implementing intermittency in an SNN. Finally, in Phase 3, the insights gained from Phases 1 and 2 are used to develop a comprehensive ImC strategy.

Overall, this approach follows a bottom-up methodology, starting with a low level of abstraction and progressively moving towards higher levels of abstraction.

4.1 Phase 1 - ImC SNN Dynamics

Based on the literature in Chapter 3, checkpointing is identified as the most relevant ImC method for our research. The primary objective of checkpointing is to regularly save the model's state, allowing the system to resume inference from the last checkpoint in the event of a power loss.

To assess the feasibility of integrating checkpointing into an SNN and identify which dynamics within SNNs are crucial for this process, the following approach is taken: First, a critical component analysis is conducted to explore the general structure of an SNN and identify the key components necessary for checkpointing. A critical component is defined as a part of the SNN architecture that is vital for creating a checkpoint. Next, each essential component is analyzed in detail, and the impact of the SNN architecture

on these components is examined. This leads to a general definition of what constitutes a checkpoint in an SNN. Following this, a method for calculating checkpoint overhead is investigated. Finally, the checkpointing process is validated, confirming the feasibility of an intermittent SNN.

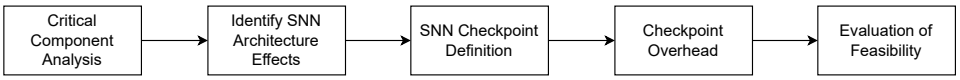


Figure 4.2: Overview of methodology within Phase 1

4.1.1 Critical Component Analysis

The first step in analyzing the critical components of an SNN within an ImC context is to identify which parts of the SNN are most relevant. This is accomplished by defining the scope and then examining the fundamentals of both SNNs and ImC.

This project focuses on the inference phase of a supervised SNN, making the forward pass of the SNN the primary area of interest. The general structure of an SNN within this context can be described as follows:

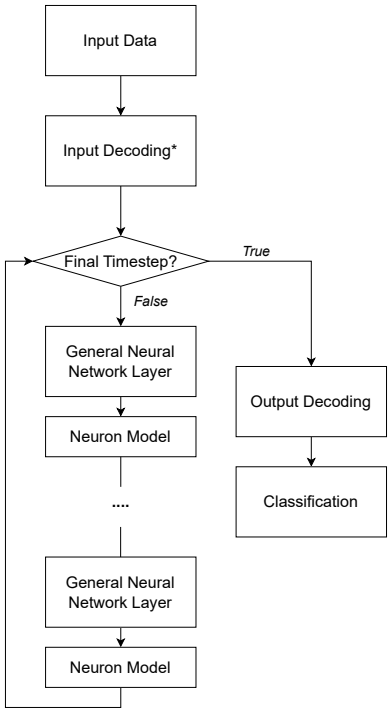


Figure 4.3: General structure of inference of an SNN, assuming data needs to be input decoded

Encoded data is processed through a general neural network layer, such as a convolutional layer. This is followed by a neuron model, which replaces the traditional activation function. After the final layer, the resulting spikes are decoded to produce the final classification.

The focus of this project is on evaluating the feasibility of checkpointing rather than

improving an existing checkpointing strategy. The general structure of checkpointing within the forward loop of a neural network can be described as follows:

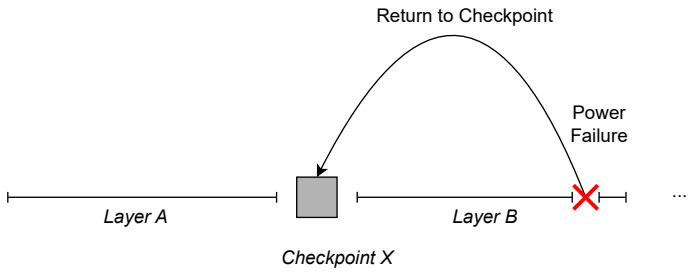


Figure 4.4: General representation of the checkpointing principle

The model's state is saved at specific points, allowing it to resume inference from the latest checkpoint rather than restarting from the beginning in the event of a power failure.

Given that our focus is on the forward pass during inference in a supervised model, we assume that the model's weights remain constant. Additionally, we assume the input data is already encoded and that inference is performed on a single batch. Since checkpoints only need to save state changes, we can disregard input encoding and the general neural network layers (model weights) and instead focus on the neuron model, timesteps, and output decoding.

Neuron Model
Section 2.4.2 describes the different neuron models and their functionalities. To limit the scope of this project, only two neuron models are considered:

- Leaky Integrate-and-Fire (LIF)
- Synaptic Conductance-Based Leaky Integrate-and-Fire (Synaptic-LIF)

The key difference between these two models lies in their complexity: the synaptic-LIF model is a more advanced neuron model compared to the standard LIF model. While the LIF model only considers a neuron's membrane potential, the synaptic-LIF model also accounts for the synaptic currents between neurons. As a result, each LIF neuron model processes two parameters—membrane potential and spike trains—whereas the synaptic-LIF model includes an additional third parameter, the synaptic current. Additionally, both membrane potential and synaptic current are associated with a decay rate. The threshold of a neuron model, which determines the membrane potential level at which the neuron fires and outputs a spike, is another crucial aspect

Within a neuron model, the decay rates and thresholds are considered hyperparameters, which are either set manually or trained together with the model weights. In other words, these parameters are set during training, do not change during inference, and thus do not need to be saved within a checkpoint.

On the other hand, membrane potentials, spike trains and synaptic currents continuously change during the forward pass. This means that these three parameters do need to be saved within a checkpoint.

Timesteps
Timesteps are discrete moments in time when the network's activities are updated. While traditional neural networks process information instantaneously, SNNs process information in intervals, simulating how neurons in the brain fire spikes of activity at separate times.

Timesteps help model the timing of spikes and the network's dynamic behaviour. In practice, a timestep of X means that the forward pass of the input data takes place X times, and the classification is only made after the final timestep.

During each forward pass, or in other words, a singular timestep, the input data, which is converted into spikes goes through all layers. If a layer receives spikes from a previous layer, it increases the membrane potential and synaptic current because of this spike. It decides to fire an output spike, only if the membrane potential is above a certain threshold.

We defined in 4.1.1 that within an SNN, the membrane potentials, synaptic currents (for Synaptic-LIF neuron models) and spiketrains change during a forward pass.

Since a timestep represents one forward pass through the model, increasing the number of timesteps means that the membrane potentials, synaptic currents, and spike trains change multiple times. During the first timestep, the model layers' membrane potentials and synaptic currents remain unchanged because the data has not yet passed through them. Therefore, there is no need to save these variables within a checkpoint at this stage. However, after the first timestep, the states of these variables have been altered, making it necessary to save them in subsequent checkpoints. Conversely, in the final checkpoint, saving these variables is unnecessary since the data will not pass through the model again.

Additionally, it is essential to consider the input data itself. Once converted into the spike domain, the data propagates as spikes firing through each layer. As a result, a checkpoint must also save the latest spike train, as this represents the input data in its most recent state.

Output Decoding
Output decoding transforms the spike-based data into a final classification. The method used for this classification depends on the type of output decoding employed, as does the timing of when the classification occurs.

Since this parameter changes during the forward pass, it must be saved within a checkpoint. To narrow the project's scope, we will focus on two types of output decoding: rate decoding and temporal decoding.

Overview of Critical Components
The analysis in the previous sections concluded that the following SNN components are essential for checkpointing:

Table 4.1: Overview of Identified Essential Components

Parameter	Options to Explore
Neuron Model	LIF, Synaptic LIF
Timesteps (ts)	ts = 1st, ts = last, ts > 1
Output Decoding	Rate Decoding, Temporal Decoding

4.1.2 SNN Architecture Effects

The results from section 4.1.1 show that the neuron model, timesteps and output decoding are relevant to checkpointing. Each type of these parameters results in a different variable that must be saved within a checkpoint. Using general knowledge regarding SNNs from section 2.4 and 3.2, the following variables are identified:

Table 4.2: Overview of model parameters and variables that were identified

Parameter	*Option*	*Variables*
Neuron Model	LIF	Membrane Potentials (Mems)
Neuron Model	Synaptic	Synaptic Currents (Syns), Mems
Output Decoding	Rate	$spk_z \cdot Timestep(ts)$
Output Decoding	Temporal	$spk_z \cdot ts_{cl}$
Timestep	$ts = 1$ or $ts = last$	Dominant $Spk_{cp}*$
Timestep	$ts > 1$	Spk_{cp}

The variables in Table 4.2 show which specific things to save within a checkpoint for a certain part of an SNNs architecture (i.e. Neuron Model, LIF). These variables are selected based on the observations made in the critical component analysis. Other parts of the SNN architecture are not considered, as we found in the critical component analysis that they did not change during inference and thus were irrelevant. The different parameters in table 4.2 can be each considered a block that is part of an 'SNN Stack', where combining these blocks (together with the parameters that were deemed unnecessary for checkpointing) results in a different SNN architecture.

The synaptic neuron model saves both synaptic currents (Syns) and membrane potentials (Mems), while the LIF neuron model saves only the membrane potentials. The key difference between the two types of output decoding lies in their updating processes: rate decoding updates the final output spike, spk_z, at every timestep, while temporal decoding follows a similar process but only up until the classification timestep,ts_{cl}. In contrast, rate decoding continues through all timesteps.

At the first or last timestep, there is no need to save anything beyond the state of the input data at that checkpoint (spk_{cp}). The concept of stacking variables, similar to how parameters are stacked to build an SNN architecture—becomes irrelevant at these points. We refer to this as a 'dominant spk_{cp}'. However, at any other timestep, the stacking principle still applies, and the current state of the input data spk_{cp} must be added to the rest of the variable stack.

Given that Table 4.2 defines each parameter option as comprising different variables that need to be saved, we can infer that different model stacks will lead to different types of checkpoints:

Table 4.3: Overview of checkpointing variable stacks for different architectures

Architecture	*Model Stack*	*Checkpointing Variables Stack*
1	Any, ts=1 or ts=last	spk_{cp}
2	LIF, Rate , ts>1	$spk_{\text{cp}}, \text{mem}_a \ldots \text{mem}_z, spk_z \cdot ts$
3	LIF, Temporal , ts>1	$spk_{\text{cp}}, \text{mem}_a \ldots \text{mem}_z, spk_z \cdot ts_{cl}$
4	Synaptic, Rate, ts>1	$spk_{\text{cp}}, \text{mem}_a \ldots \text{mem}_z, \text{syn}_a \ldots \text{syn}_z, spk_z \cdot ts$
5	Synaptic, Temporal, ts>1	$spk_{\text{cp}}, \text{mem}_a \ldots \text{mem}_z, \text{syn}_a \ldots \text{syn}_z, spk_z \cdot ts_{cl}$

4.1.3 Checkpoint Overhead

Table 4.3 identified the types of checkpoints for different model architectures. However, the overhead of a checkpoint is an essential factor to consider when evaluating its performance. A checkpoint is essentially an array of variables, and thus, the overhead of a

checkpoint is the act of writing and reading this array. In ImC, this act of reading and writing checkpoints is mostly with Non-Volatile Memory (NVMe) [18]. In NVMe, the total overhead in both latency and energy consumption can be calculated by multiplying the total amount of bytes written or read by the overhead per singular byte.

Spike trains are stored as binary variables, represented by either a 1 or 0, and are therefore saved as integers. In contrast, synaptic currents and membrane potentials are non-integer values and are stored as floats (e.g., float16). Based on these observations and the results in Table 4.3, a general formula for checkpoint overhead in bytes can be defined:

$$Overhead_{cp} = \frac{spk_{cp} \cdot D + spk_z \cdot ts}{8} + \frac{D \cdot N_m}{8} \times \sum_{layer=a}^{z} (L_n) \tag{4.1}$$

In which:

- $Overhead_{cp}$: Total overhead in bytes for a specific checkpoint
- spk_{cp}: Size of the spike train within the checkpoint layer
- spk_z: Size of the spike train in the last layer
- ts: Amount of timesteps (>1)
- D: Amount of bits within mem and syn datatype
- N_m: Amount of variables within the neuron model
- L_n: Total amount of output neurons in the layer
- $layer = a$ to z: All layers within the model

Equation 4.1 has an exception at the first and last timesteps of the model:

$$Overhead_{cp} = \frac{spk_{cp} \cdot D}{8} \tag{4.2}$$

In which:

- $Overhead_{cp}$: Total overhead in bytes of a checkpoint when the model is either at the first or last timestep
- spk_{cp}: Size of the spike train within the checkpoint layer
- D: Amount of bits within mem and syn datatype

However, it is important to note that some models operate with a single timestep, which is another use-case of equation 4.2.

To figure out which components within equation 4.1 are dominant, a sensitivity analysis can be performed. The following base magnitudes were used for this sensitivity analysis of the equation:

The magnitude of each of these variables is either doubled or halved, and the resulting overhead is compared with the base model. Using this method, we can see how much each variable within 4.1 affects the total overhead. The normalized results are:

Table 4.4: Identified variables and their magnitude used within the sensitivity analysis

Variable	*Magnitude*
spk_{cp}	500
spk_z	10
ts	25
D	16
N_m	1
L_n	1500

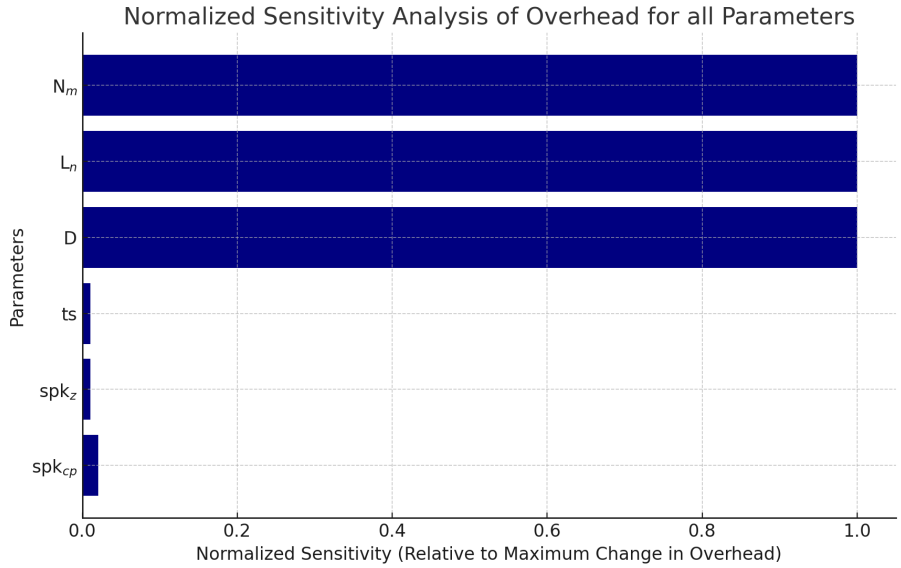


Figure 4.5: Sensitivity analysis results

The results of the sensitivity analysis show that N_m, L_n and D have the most significant impact on total checkpoint overhead.

4.1.4 Evaluation of Feasibility

To validate the feasibility of checkpointing, each model architecture and its corresponding checkpointing variable stack (Table 4.3) must be evaluated. We accomplished this by using pre-trained models that align with the architectures in Table 4.3 and verifying that their output accuracy remained unchanged. The results indicate that our methodology did not significantly impact model accuracy, confirming its feasibility.

4.2 Phase 2 - ImC SNN vs SNN Trade-offs

In the following section, we will examine the trade-offs between an intermittent SNN and a non-intermittent SNN. However, to define this trade-off, we need to establish the factors for evaluating an intermittent SNN and develop methods for measuring its performance. First, we need to create a general methodology for evaluating and comparing intermittent and non-intermittent SNNs. Next, we need to establish a process for calculating the overall checkpoint overhead, followed by a comparable approach for determining the inference overhead.

4.2.1 Checkpoint Location

As the checkpoint location has not yet been defined, we will evaluate all options for potential checkpoint locations. A potential spot of the checkpoint is after a model layer (i.e. checkpoint 1 is after layer 1). Inter-layer checkpoints will not be considered in this project.

4.2.2 Energy Evaluation

To evaluate if an intermittent SNN is worth it compared to a non-intermittent SNN, the total energy consumption of an intermittent SNN has to be compared to the total energy consumption of a non-intermittent SNN. There is no need to checkpoint if the total non-intermittent energy consumption, or $E_{inference}$, is higher than the intermittent energy consumption. In that case, it makes more sense to run inference from the beginning after system power loss.

The total energy consumption of an intermittent SNN consists of two parts. First, the energy consumption of the checkpoint itself, E_{cp}, and second, the energy consumption to finish inference after the checkpoint. The energy consumption before the checkpoint is irrelevant, as this does not influence the reasoning checkpointing. This can be all summarized in the following equation:

The total energy consumption of an intermittent SNN comprises two components: first, the energy consumed by the checkpoint itself (E_{cp}), and second, the energy required to complete the inference after the checkpoint (E_{cont}). The energy consumed before the checkpoint is irrelevant, as it does not impact the decision to checkpoint. This relationship can be summarized by the following equation:

$$E_{cp} + E_{cont} < E_{inf} \tag{4.3}$$

In which:

- E_{cp}: Energy consumption of a checkpoint
- E_{cont}: Energy consumption to finish inference after a checkpoint
- E_{inf}: Total inference energy consumption

Checkpointing is deemed unnecessary if the sum of E_{cp} and E_{cont} exceeds E_{inf}. Alternatively, checkpointing is justified when this sum is lower than E_{inf}

4.2.3 Latency Evaluation

The same logic in Equation 4.3 cannot be applied to latency evaluation. In an intermittent SNN, higher latency compared to a non-intermittent SNN does not necessarily mean that checkpointing is not worth it. This is especially relevant because latency in an intermittent SNN will inherently be higher due to the need for energy harvesting during system power failures.

However, latency can still be a useful metric for determining if critical events are missed, especially if the system is stalled while creating checkpoints. This concern is especially relevant for medical devices, where missing events could have serious consequences. Therefore, the latency of a single checkpoint (T_{cp}) should be compared against the overall inference latency (T_{inf}) to identify any potential limitations introduced by checkpointing latency.

4.2.4 Checkpoint Energy Consumption (E_{cp})

Equation 4.1 in section 4.1.3 defined the overhead of a checkpoint in Bytes. The results of this equation can be used to calculate the Energy consumption of a checkpoint in Joules:

$$E_{cp} = Overhead_{cp} \times E_{NVMe} \tag{4.4}$$

In which:

- E_{cp}: Energy consumption of a checkpoint
- $Overhead_{cp}$: Total overhead in bytes for a specific checkpoint
- E_{NVMe}: Energy consumption of NVMe for a single byte

This is the energy consumption for a single read or write to NVMe.

4.2.5 Checkpoint Latency (T_{cp})

Similarly to energy consumption, using the results of equation 4.1, the checkpoint latency can be calculated:

$$T_{cp} = Overhead_{cp} \times T_{NVMe} \tag{4.5}$$

In which:

- T_{cp}: Latency of a checkpoint
- $Overhead_{cp}$: Total overhead in bytes for a specific checkpoint
- L_{NVMe}: Latency of NVMe for a single byte

This is the latency for a single read or write to NVMe.

4.2.6 Inference Energy Consumption(E_{inf})

Due to time limitations, it was not possible to attain hardware for this project, meaning the inference energy needs to be estimated. Multiple papers in the literature describe quantitative methods for calculating the energy and latency of SNNs. Gamage et al. [26] describe summing up all spikes that triggered a Synaptic Operation (SOP) and multiplying this with the energy consumption of a single SOP:

$$E = SOP \times E_{SOP} \tag{4.6}$$

In which:

- E: Energy consumption of an SNN
- SOP: Total amount of Synaptic Operations
- E_{SOP}: Energy consumption of a single SOP

Gamage et al. [26] use the Intel Loihi SOP power consumption in their calculations and estimate that their SNN is 65-135 times more energy efficient than an Artificial Neural Network (ANN).

Lemaire et al. [27] refute this method for energy consumption calculations, as they believe that SOP play only a small part within the total energy consumption of an SNN. Lemaire et al. [27] also describe how other literature similarly doubted the methodology used in Gamage et al. [26], as the power efficiency results were deemed unreasonably high. Lemaire et al. [27] propose their own method, which supposedly is a more realistic power estimation:

$$E_{inf} = E_{mem} + E_{ops+addr} \tag{4.7}$$

In which:

- E_{inf}: Energy consumption of an SNN
- E_{mem}: Energy consumption of memory access
- $E_{ops+addr}$: Energy consumption of synaptic operations and addressing mechanisms

In this project, the methodology proposed by Lemaire et al. [27] is chosen for energy consumption estimations due to its extensive nature and relatively realistic results.

4.2.7 Inference Latency (T_{inf})

The inference energy estimation method by Gamage et al. [26] has been implemented for latency estimations as well. However, we assume that the reported unrealistic estimation of inference energy by this method most likely leads to unrealistic inference latency estimations.

Unfortunately, the method described by Lemaire et al. [27] does not account for latency. It might be possible to adjust this method to account for latency as well, however, this is outside of the scope of this project. No other viable method for estimating inference latency was found.

As an alternative, we will use the inference latency of an SNN model from the literature as T_{inf}. Ideally, this reference model should use the same dataset and have a similar architecture.

4.2.8 Continued Inference Energy (E_{cont})

Section 4.2.2 defined the continued inference overhead (E_{cont}) as the energy cost to finish inference after a checkpoint. Assuming that the total inference energy is evenly spread over all timesteps, the inference energy for a single timestep can be defined as:

$$E_{ts} = \frac{E_{inf}}{ts} \tag{4.8}$$

Where:

- E_{ts} is the total energy consumption of a single timestep
- E_{inf} is the total inference energy consumption
- ts is the number of timesteps

Using equation 4.8, the inference after a checkpoint, E_{cont}, can be calculated:

$$E_{cont} = E_{Rem} + ts_{Rem} \times E_{ts} \tag{4.9}$$

Where:

- E_{cont} is the total energy consumption after a checkpoint
- E_{Rem} is the energy consumption of the remaining layers after a checkpoint within the same timestep
- ts_{Rem} is the timesteps remaining after the checkpoint
- E_{ts} is the inference energy consumption for a single timestep

4.2.9 Overview of Energy Consumption and Latency Estimation

Table 4.5: Overview of all evaluation factors that were identified, and its corresponding evaluation method

Evaluation Factor	*Evaluation Method*
Checkpoint Energy (E_{cp})	$E_{cp} = Overhead_{cp} \times E_{NVMe}$
Checkpoint Latency (T_{cp})	$T_{cp} = Overhead_{cp} \times T_{NVMe}$
Inference Energy (E_{inf})	$E_{inf} = E_{mem} + E_{ops+addr}$
Inference Latency (T_{inf})	Literature Comparison
Continued Inference Energy (E_{cont})	$E_{cont} = E_{rem} + ts_{rem} \times E_{ts}$
Total Energy ImC SNN (E_{tot})	$E_{tot} = E_{cp} + E_{cont}$
Total Latency ImC SNN (T_{tot})	$T_{tot} = T_{inf} + T_{cp}$

4.3 Phase 3 - ImC SNN Strategy

Based on the energy and latency evaluation in 4.2, an ImC SNN strategy can be created. This consists of defining the location of checkpoints, and the frequency of checkpointing.

4.3.1 Checkpointing Location

Using the results from Equation 4.3, it is possible to estimate the optimal checkpointing locations for a specific SNN model. By applying Equation 4.3 at different checkpointing locations and timesteps, potential checkpointing points can be identified whenever $E_{cp} + E_{cont} < E_{inf}$.

4.3.2 Checkpointing Frequency

To determine the checkpointing frequency, or the number of times to create a checkpoint during a timestep or forward pass, the following method can be used :

$$E_{cp} < E_{beforecp} \tag{4.10}$$

In which:

- E_{cp}: Energy consumption of the checkpoint
- $E_{beforecp}$: Energy consumption of the model between the current checkpoint and the previous checkpoint

Equation 4.10 compares the energy consumption between the current checkpoint and the previous checkpoint $E_{beforecp}$ with the energy required for checkpointing E_{cp}. If $E_{beforecp}$ exceeds a checkpoint should be added. Otherwise, checkpointing is not beneficial as it would result in a net energy loss.

Chapter 5

SNN-EMG Case Study

In this case study, we will build upon the findings from the feasibility study and apply them to an EMG SNN model. The objective is to further evaluate the results from Chapter 4 using real-world data.

5.1 SNN Model

5.1.1 Dataset

Ninapro DB2 was selected as the dataset for the SNN model. In this dataset, surface EMG signals are recorded using 12 electrodes placed on the subject's forearm. An Inertial Measurement Unit (IMU) sensor is also placed on the subject's wrist; however, this data is not included within the SNN model.

This dataset consists of 40 patients, of whom 29 are males and 11 are females. The median age for males is 29 years ± 3.99, while the median for females is 30 years ± 2.87. In this dataset, subjects participated in three different exercises. Each exercise consisted of multiple hand/arm movements that the patient had to repeat 6 times, each lasting 5 seconds. To limit the scope and decrease time spent working on optimizing the SNN model, the first 12 movements from exercise B were chosen as the dataset of this project. Figure 5.1 shows the selected hand movements.

The distribution of samples between the 12 gestures in the data is plotted in Figure 5.2. There is a slight imbalance in the distribution of samples among gestures. Gesture 1 is overrepresented, while gestures 6 and 8 are underrepresented.

5.1.2 Preprocessing

The process behind the data acquisition of the Ninapro DB2 dataset is described by Atzori et al.[28]. This dataset is not raw EMG data but has instead been filtered to remove power-line interference using a Hampel filter. Furthermore, the data streams of all electrodes were synchronised using high-resolution timestamps.

The dataset goes through a preprocessing pipeline outlined in Figure 5.3. As mentioned in Section 5.1.1, the first twelve gestures are selected, and any resting periods between gestures are eliminated so that only the gestures themselves are included in the dataset. Finally, the samples are divided into segments of 200 ms, following the suggestion of Atzori et al. [28].

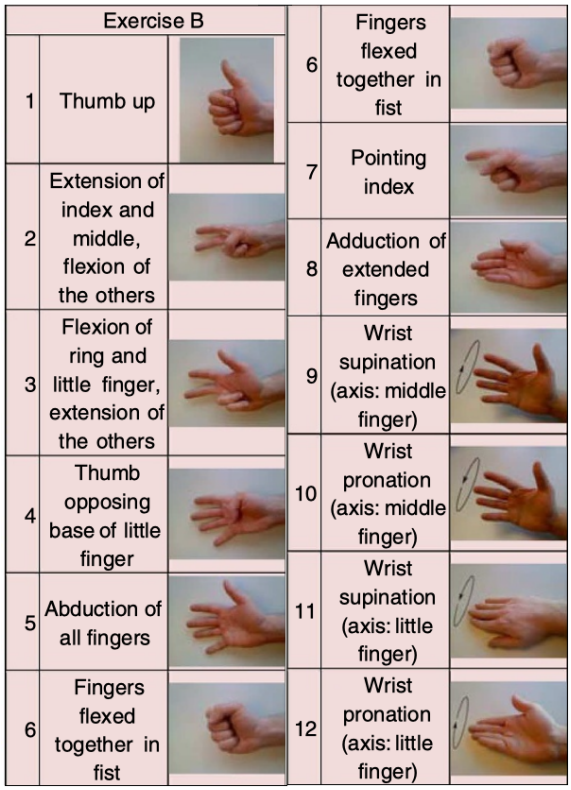


Figure 5.1: Overview of the selected hand movements

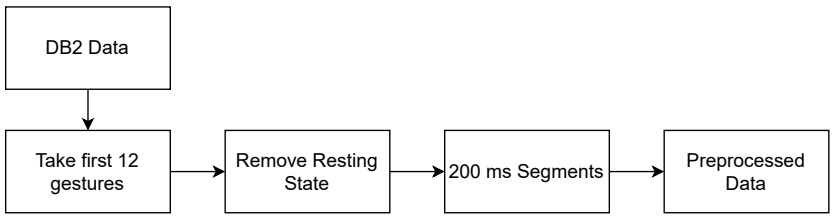


Figure 5.3: Proposed preprocessing pipeline

5.1.3 Multi People vs Single Person EMG Model

The Ninapro DB2 dataset consists of sub-datasets, each representing a specific patient. This differs from other biomedical datasets, such as electroencephalogram (EEG), where all patient data is combined into a single dataset. Separating patients allows for patient-specific model training, which is useful, for example, in robotic prosthetics.

An argument can be made that the general approach, where all individual datasets are combined into a general dataset, is the correct method for this project since it is not patient-specific. However, a significant counterpoint would be that creating a general dataset would also result in a much bigger dataset, which leads to a much longer time to train the model. This is impractical, and since the main objectives of this project are regarding the intersection of intermittent computing and SNNs, a general dataset is considered unnecessary. Instead, the model will be based on the data of a randomly selected patient.

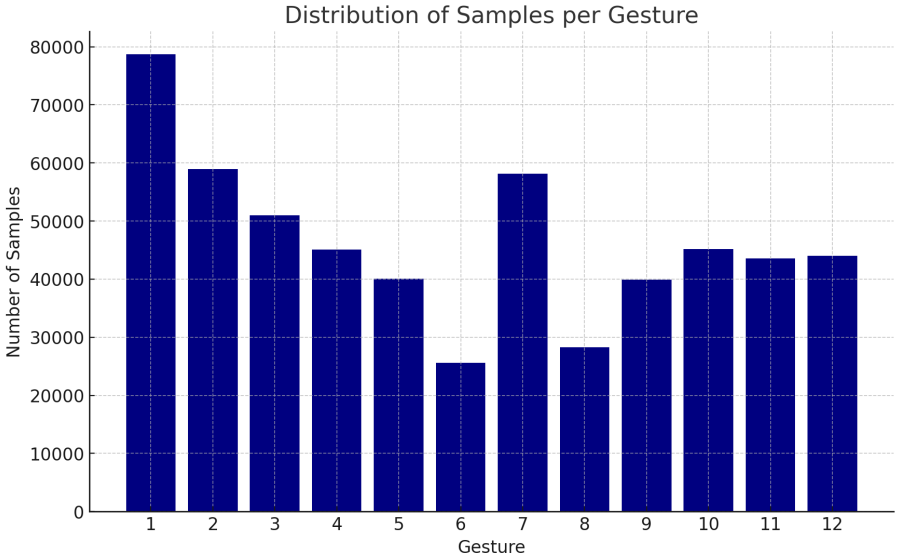


Figure 5.2: Distribution of samples between gestures

5.1.4 SNN Platform

This project was accepted by the Intel Neuromorphic Research Community, which should have given us access to the Intel Loihi Neuromorphic Chip. Unfortunately, we could not figure out the legalities of this agreement with Intel in time. As a result, this project was completed without the use of neuromorphic hardware. We chose SNNTorch as the platform to train and evaluate the SNN. SNNTorch was selected over other platforms, such as SpikingJelly or Norse, because of its extensive documentation.

5.1.5 Input Encoding

Input Encoding in SNNs is a critical research area [29][30], as the choice of encoding directly impacts both the accuracy and power efficiency of the network. In this project, we focus exclusively on the influence of input encoding on accuracy, deliberately excluding considerations of its effect on ImC due to time constraints.

The literature we found lacks a standardized methodology for evaluating the performance of different input encoding schemes. However, our observations during model optimization suggest that the initial performance of an untrained model can significantly vary depending on the chosen input encoding. This implies that selecting an appropriate encoding scheme is a crucial step that should precede model training.

In this study, we explored three encoding methods: rate encoding, temporal encoding, and delta encoding. For the Ninapro DB2 dataset, rate encoding demonstrated the highest performance, followed by delta encoding. This indicates that rate encoding may be more suitable for this specific dataset and task, though further exploration and validation are necessary to generalize these findings.

5.1.6 Model Architecture

This project aims to better understand the applicability of intermittent behaviour within SNNs. To keep the project scope manageable, the goal for the SNN model to be developed is to achieve a test accuracy of at least 55%. In section 3, we found that non-SNN models that used this dataset had accuracies ranging from 60.27% to 82.2%.

The development of the SNN model began with a review of the architectures used in similar models, as described in Chapter 3. This was followed by an iterative process of

experimenting with different architectures.

Ultimately, the following architecture was selected:

Table 5.1: Proposed model layers and their corresponding size

Layer	*Size*
Layer 1 - Convolution	64 Filters, kernel size (60,1)
Flattening	-
Layer 2 - Fully Connected	256 Units
Layer 3 - Fully Connected	128 Units
Layer 4 - Fully Connected	12 Units

The neuron model used in our SNN was a synaptic-LIF, with ten timesteps and rate decoding employed as the output decoding method.

5.1.7 Model Performance

The final model was evaluated over 100 epochs and achieved a testing accuracy of 60.76%, as illustrated in Figure 5.4.

The confusion matrix in Figure 5.5 reveals that classes 0 (gesture 1), 6 (gesture 7), and 11 (gesture 12) demonstrate relatively strong performance, with minimal misclassification.

Conversely, classes 1 (gesture 2), 4 (gesture 5), and 8 (gesture 9) exhibit poorer performance, with a higher degree of misclassification. This could be attributed to the similarity between certain gestures, leading to confusion by the model.

Interestingly, classes 5 (gesture 6) and 7 (gesture 8) did not perform as poorly as expected despite having the fewest samples in the dataset. This suggests that the quantity of data may not significantly influence model performance as much as the complexity or similarity of the gesture types, indicating that some gestures are inherently more challenging to classify.

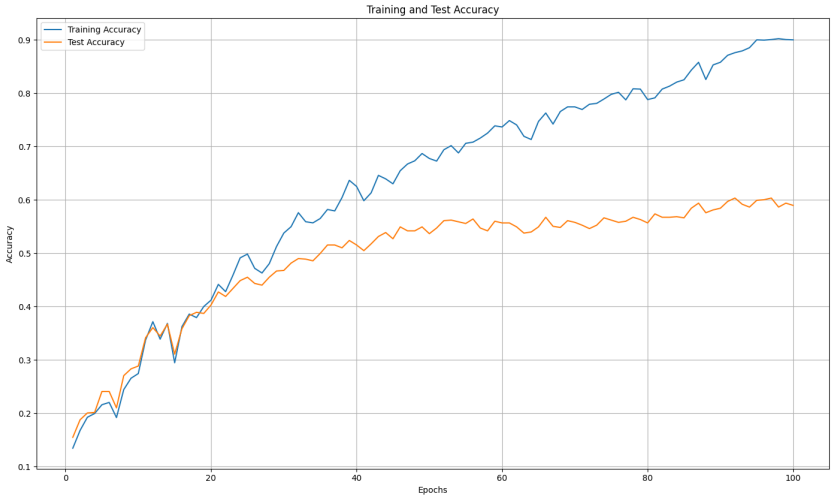


Figure 5.4: Train and test accuracy of the SNN model over 100 epochs

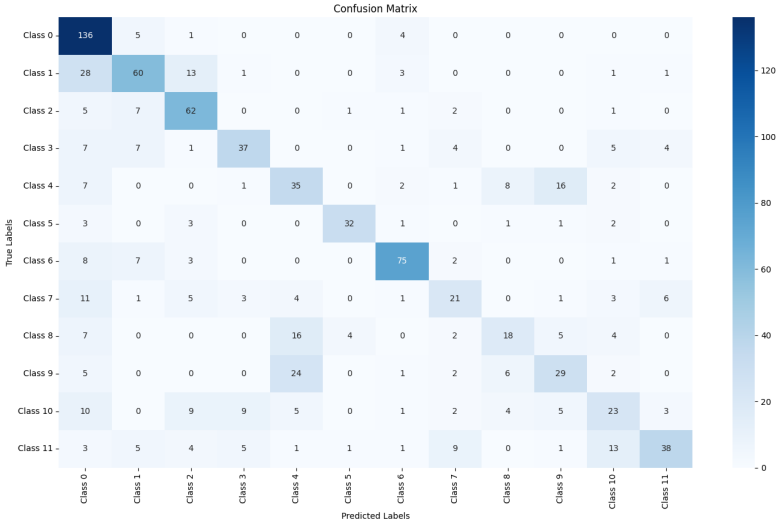


Figure 5.5: Confusion matrix, representing misclassifications of the SNN model

5.2 Intermittent EMG SNN

5.2.1 Checkpointing Overhead

In Section 5.1.6, the SNN model architecture was defined. Using Equation 4.1, the checkpoint overhead for our SNN model can be computed:

Table 5.2: Overhead of each checkpoint

Checkpoint	*Overhead (Bytes)*
1	4797
2	4733
3	4717
4	4703

The overhead for each checkpoint is computed, with checkpoints placed sequentially after each model layer (e.g., checkpoint 1 follows the convolutional layer, and checkpoint 2 follows the first fully connected layer). Overall, the difference in overhead between checkpoint 1, which has the highest overhead, and checkpoint 4, which has the lowest, is relatively minor, amounting to approximately 1.96%.

Using the methodology outlined in Section 4.2.4 and Equation 4.1, the general overhead in bytes can be converted into energy overhead in joules. Equation 4.4 requires the energy needed to write or read a single byte to NVMes. Since we did not have access to hardware during this project, we selected NVMes based on criteria we would use when designing a system. We began by defining the device requirements, which, in our case, include sufficient capacity to store our checkpoints and maximise energy efficiency. We then selected an NVMes device by reviewing datasheets for various devices that meet these requirements.

Table 5.2 shows that the largest checkpoint size we need to store is 4797 bytes, which sets the minimum storage capacity for the NVMe device we select. After exploring various types of NVMe devices, we identified FRAM NVMe devices as particularly energy-efficient, often used in ImC applications [18]. We searched different electronics distributors and found the Fujitsu MB85RC256V FRAM module [31], which offers a total capacity of 32 kB, sufficient for our needs. Using the datasheet of the MB85RC256V FRAM module, we can calculate E_{NVMe}:

Given:

- Operating Voltage, $V = 5.5\,\mathrm{V}$
- Maximum Operating Current, $I = 200\,\mu\mathrm{A} = 200 \times 10^{-6}\,\mathrm{A}$
- Clock Frequency, $f = 1\,\mathrm{MHz} = 1 \times 10^{6}\,\mathrm{Hz}$
- Bits per Byte = 8

$$t_{\text{bit}} = \frac{1}{f}$$

$$t_{\text{bit}} = \frac{1}{1 \times 10^{6}\,\mathrm{Hz}} = 1 \times 10^{-6}\,\mathrm{s/bit}$$

Energy per Bit (E_{bit}):

$$E_{\text{bit}} = V \times I \times t_{\text{bit}}$$

$$E_{\text{bit}} = 5.5\ \times 200 \times 10^{-6}\ \times 1 \times 10^{-6}\,\mathrm{s}$$

$$E_{\text{bit}} = 1.1 \times 10^{-9}\,\mathrm{J/bit}$$

Energy per Byte (E_{byte}):

$$E_{\text{byte}} = E_{\text{bit}} \times 8$$

$$E_{\text{byte}} = 1.1 \times 10^{-9}\,\mathrm{J/bit} \times 8$$

$$E_{\text{byte}} = 8.8 \times 10^{-9}\,\mathrm{J/byte}$$

$$E_{\text{byte}} = 0.0088\,\mu\mathrm{J/byte}$$

Time per Bit (t_{bit})

$$t_{\text{bit}} = \frac{1}{f}$$

$$t_{\text{bit}} = \frac{1}{1 \times 10^{6}\,\mathrm{Hz}} = 1 \times 10^{-6}\,\mathrm{s/bit}$$

Time per Byte (t_{byte})

$$t_{\text{byte}} = t_{\text{bit}} \times 8$$

$$t_{\text{byte}} = 1 \times 10^{-6}\,\text{s/bit} \times 8$$

$$t_{\text{byte}} = 8 \times 10^{-6}\,\text{s/byte}$$

Using the calculated values of E_{NVMe} and T_{NVMe}, along with Equation 4.4, we compute E_{cp} and T_{cp}:

Table 5.3: Overhead in E and T for each checkpoint

Checkpoint	E_{cp} *(uJ)*	T_{cp} *(ms)*
1	42.2	38.4
2	41.7	37.9
3	41.5	37.7
4	41.4	37.6

5.2.2 Architecture Checkpoint Overhead Comparison

Table 5.2 outlines the checkpoint overhead in bytes at various locations within our SNN model, as discussed in Section 5.1.6. Our SNN model most closely aligns with architecture 4 in Table 4.3. We apply the same model layers detailed in Table 4.3 to estimate the overhead for other architectures listed in Table 5.1. This approach provides an estimation of the overhead for each architecture type:

Table 5.4: Overhead in bytes for different architectures

Checkpoint	*Overhead (Bytes) Architecture 1*	*Overhead (Bytes) Architecture 2*	*Overhead (Bytes) Architecture 3*	*Overhead (Bytes) Architecture 4/Base*	*Overhead (Bytes) Architecture 5*
1	768	2439	2439*	4797	4797*
2	256	2375	2375*	4733	4733*
3	128	2359	2359*	4717	4717*
4	12	2345	2345*	4707	4707*

Using Equation 4.4, the energy consumption for each checkpoint across different architectures can be calculated as follows:

Table 5.5: Overhead in terms of energy and latency for different architectures

Checkpoint	E *(uJ) Archt.1*	T *(ms) Archt.1*	E *(uJ) Archt.2*	T *(ms) Archt.2*	E *(uJ) Archt.3*	T *(ms) Archt.3*	E *(uJ) Archt.4*	T *(ms) Archt.4*	E *(uJ) Archt.5*	T *(ms) Archt.5*
1	6.76	6.14	21.5	19.5	21.5*	19.5*	84.4	76.8	84.4*	76.8*
2	2.25	2.05	20.9	19.0	20.9*	19.0*	83.4	75.8	83.4*	75.8*
3	1.13	1.02	20.8	18.9	20.8*	18.9*	83.0	75.4	83.0*	75.4*
4	0.106	0.096	20.6	18.8	20.6*	18.8*	82.8	75.2	82.8*	75.2*

The results in Table 5.5 indicate that architectures 4 and 5 have the highest energy consumption and latency. In contrast, architecture 1 exhibits significantly lower overhead

compared to the other architectures. It is important to note that architectures 3 and 5 utilize temporal decoding, meaning that all overhead calculations assume the worst-case scenario, where classification occurs at the final timestep.

5.2.3 Inference Energy

In Section 4.2.6, we adopted the method proposed by Lemaire et al. [27] to estimate the inference energy consumption of our SNN model. This approach calculates the energy consumption for each model layer individually and then sums these values to determine the total inference energy consumption. The detailed calculation of inference energy consumption is provided in Appendix A A. The results for each layer are as follows:

Table 5.6: Total energy for each model layer in 5.1

Layer	E_{mem} *(uJ)*	$E_{ops+addr}$ *(uJ)*	*Total Energy (uJ)*
1	401.41	0.30	401.70
2	194.27	4.21	198.48
3	37.52	3.20	40.72
4	2.25	0.094	2.34

We assume that the average energy per timestep remains constant. Thus, if the total energy E_{total} is known, the energy per timestep can be calculated as $E_{total}/total\ timesteps$.

Using this assumption and the results from Table 5.6, we can determine the continued inference energy,E_{cont}, as defined in Equation 4.9 in Section 4.2.8:

Table 5.7: Continued energy consumption, E_{cont}, for each checkpoint over all 10 timesteps

Timesteps	E_{cont} *Checkpoint 1*	E_{cont} *Checkpoint 2*	E_{cont} *Checkpoint 3*	E_{cont} Checkpoint 4
1	603.07	583.22	579.15	578.92
2	538.75	518.90	514.83	514.59
3	474.42	454.57	450.50	450.27
4	410.10	390.25	386.18	385.94
5	345.77	325.93	321.85	321.62
6	281.45	261.60	257.53	257.30
7	217.13	197.28	193.21	192.97
8	152.80	132.95	128.88	128.65
9	88.48	68.63	64.56	64.32
10	24.154	4.306	0.234	0.00

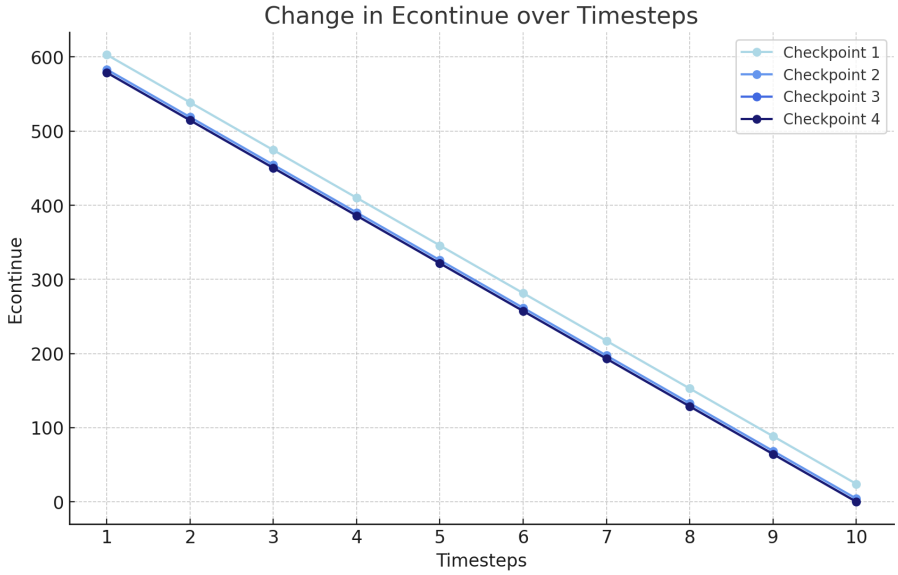


Figure 5.6: Change of E_{cont} over 10 timesteps

5.2.4 Inference Latency

In Section 4.2.7, we noted the absence of a method to accurately calculate the inference latency of our model. Consequently, we proposed using inference latency data from existing literature as a reference point. However, we could not find any spiking neural network (SNN) models for EMG data that also utilized the Ninapro DB2 dataset. To address this, we expanded our search to include studies using different datasets but with similar model architectures.

Vitale et al. [14] developed both a convolutional SNN and a fully connected SNN using the Ninapro DB5 dataset. They reported an inference latency of 5.7 ms, which we will use as an estimate for our model's inference latency. This value represents the total inference latency, which means that the latency per timestep can be calculated by dividing this by the total amount of timesteps, 600 in this case, which results in a latency per inference timestep, T_{TS}, of 0.0095 ms. Furthermore, we have to assume that the latency in each checkpoint is the same since we have no method to calculate latency for individual checkpoints.

5.2.5 Checkpointing Feasibility - Energy

The feasibility of checkpointing regarding energy was established using Equation 4.3. Applying this methodology, along with the results from Sections 5.2.1 and 5.2.3, allows us to assess the feasibility of checkpointing within our SNN model. By combining checkpoint energy consumption,E_{cp}, with continued inference energy consumption, E_{cont}, we obtain the following results:

Table 5.8: Econt+Ecp for each checkpoint over 10 timesteps

Timestep	*Econt+Ecp Checkpoint 1*	*Econt+Ecp Checkpoint 2*	*Econt+Ecp Checkpoint 3*	*Econt+Ecp Checkpoint 4*
1	609.83	585.47	580.28	579.02
2	580.95	560.60	556.33	555.99
3	516.62	496.27	492.00	491.67
4	452.30	431.95	427.68	427.34
5	387.97	367.63	363.35	363.02
6	323.65	303.30	299.03	298.70
7	259.33	238.98	234.71	234.37
8	195.00	174.65	170.38	170.05
9	130.68	110.33	106.06	105.72
10	30.90	6.56	1.36	0.106

Plotting the remaining timesteps against $E_{continue}$+E_{cp} results in the following graph:

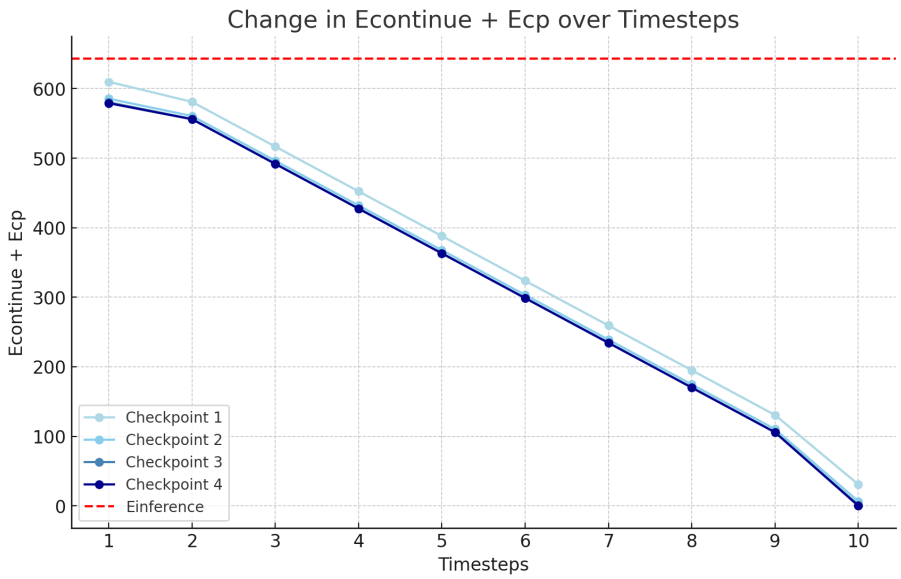


Figure 5.7: Combined energy consumption of E_{cp} and E_{cont} over 10 timesteps

The total inference energy is represented by a dotted red line. Any checkpoint falling below this line meets the condition specified in Equation 4.3 and is therefore deemed worthwhile for checkpointing.

5.2.6 Checkpointing Feasibility - Latency

Using the results in section 5.2.4 and 5.2.2, we can evaluate the effect of checkpointing on the total latency of the system.

In comparison to the inference latency (T_{inf}) of 0.0095 ms, the checkpointing latency (T_{cp}) is significantly higher at 38.4 ms. As shown in Figure 5.8, this disparity causes the total latency to increase substantially over ten timesteps, reaching 307.74 ms. This represents a significant increase in overall system latency, driven by the checkpointing process.

Table 5.9: Change in inference latency, and combined inference and checkpoint latency over ten timesteps

Timestep	*Inference Latency (us)*	*Inference + CP Latency (us)*
1	9.50	15.64
2	19.00	38434.64
3	28.50	76863.14
4	38.00	115301.14
5	47.50	153748.64
6	57.00	192205.64
7	66.50	230672.14
8	76.00	269148.14
9	85.50	307633.64
10	95.00	307734.79

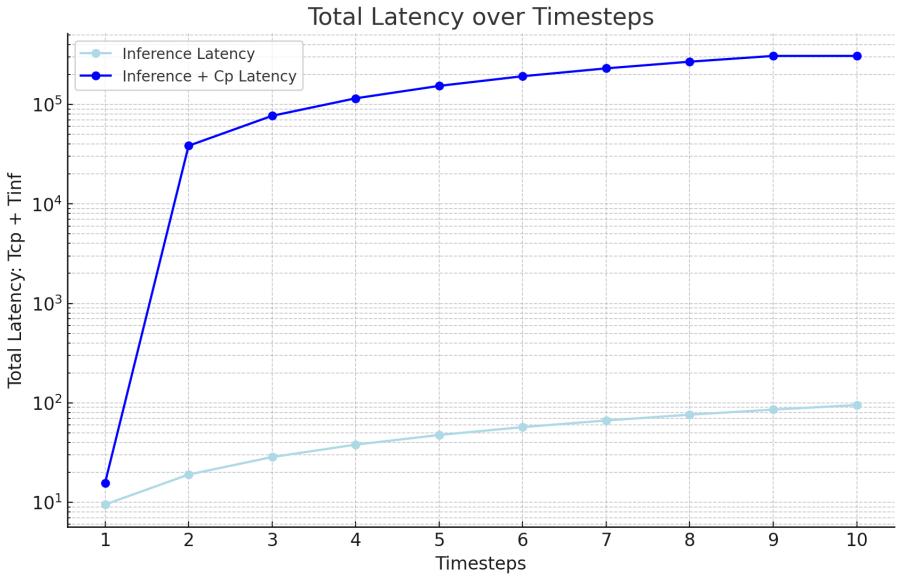


Figure 5.8: Change in inference latency and total latency over ten timesteps

5.2.7 Checkpointing Strategy

Based on the results of the checkpointing feasibility calculations in section 5.2.5, it is possible to create a checkpointing strategy for our SNN model. The results in Table 5.8 show that it is not always worth checkpointing at early timesteps. However, at higher timesteps, the potential energy savings by a checkpoint increase substantially. For our specific model, the strategy would be:

Based on the checkpointing feasibility calculations in Section 5.2.5, we can develop a checkpointing strategy for our SNN model. The results in Table 5.8 indicate that checkpointing may not always be beneficial at early timesteps. However, as timesteps increase, the potential energy savings from checkpointing rise significantly. For our specific model, the recommended strategy would be:

- For all checkpoints, it is advantageous to checkpoint at every timestep
- At earlier timesteps, total potential energy gain is much less than in later timesteps

5.2.8 Checkpointing Frequency

Section 4.3.2 introduced Equation 4.10 to determine the optimal checkpointing frequency during each forward pass (or timestep). Using the findings from Sections 5.2.1 and 5.2.3:

Table 5.10: Testing if increasing checkpointing frequency is worth it for all four checkpoints

Checkpoint	E_{cp}	$E_{beforecp}$	$E_{cp} < E_{beforecp}$
1	42.2	40.41	False
2	41.7	19.85	False
3	41.5	4.72	False
4	41.4	0234	False

The results in Table 5.10 indicate that in our SNN model, multiple checkpoints within a single forward pass (or timestep) are not beneficial.

Chapter 6

Discussion

6.1 General ImC SNN

In Chapter 4, we demonstrated the feasibility of implementing intermittent SNNs using checkpointing techniques. We also introduced a method for quantifying checkpoint overhead in terms of both accuracy and latency. Applying this methodology to our SNN EMG model, as discussed in Chapter 5, we assessed the energy and latency overheads specific to this model. Subsequently, we utilized the feasibility criterion from Equation 4.3 to determine the optimal timesteps and checkpoints for implementing checkpointing. Our analysis indicated that for our SNN model, checkpointing is advantageous at all evaluated checkpoints.

However, the analysis reveals a distinct pattern: the potential energy savings from checkpointing are minimal at earlier timesteps. For instance, checkpointing at timestep one yields only a marginal reduction in total energy overhead compared to rerunning inference from the start after a power failure. This observation suggests that a dynamic checkpointing strategy should be implemented for intermittent SNNs employing checkpointing, where the frequency of checkpointing adapts based on the current timestep. This approach aligns with the dynamic checkpointing frequency algorithm proposed by Maeng et al.[21]. Different model types are expected to result in different checkpointing frequencies and locations.

Additionally, our findings in Section 4.1.3 indicate that the layer size primarily influences checkpoint overhead. For example, a convolutional layer with 64 filters incurs significantly higher checkpoint overhead than one with 32 filters. Conversely, the overhead difference between models with one timestep and those with 100 timesteps is negligible.

6.2 ImC SNN EMG

In Chapter 5, we determined that the worst-case scenario for total energy and latency overheads in our system were 609.83 µJ and 307.73 ms, respectively, resulting in a power requirement of 187.67 µW. Literature on the energy harvesting capabilities of devices suggests that the available power typically ranges from micro to milliwatts [32] [33]. Given this, the power requirements of our SNN EMG model appear within a feasible range, making it plausible from an energy standpoint.

However, the latency observed is considerably high, which poses a significant concern given that our application is intended for medical use. Such high latency could result in the loss of critical input data, rendering the system unsuitable for many medical applications.

Nevertheless, it may be possible to mitigate this issue by selecting a different type of NVMe, potentially reducing the latency to acceptable levels.

6.3 Future Works

This project focused on the early development stages of an intermittent SNN. However, several aspects of this topic remain unexplored. Our results suggest that different SNN architectures may necessitate distinct checkpointing frequencies and locations. We hypothesize that each architecture has its optimal "sweet spot" for checkpointing. This hypothesis requires further investigation through the design and testing of additional architectures to confirm its validity.

Moreover, this project's scope was intentionally limited to a narrow range of SNN architectures. To determine our findings' generalizability, exploring other architectures, such as SNNs employing the Integrate-and-Fire neuron model and population decoding mechanisms is essential.

Another important consideration is the impact of various ImC dynamics, such as capacitor size and the characteristics of energy harvesting devices, within the context of intermittent SNNs. Developing an energy harvesting forecasting model akin to the approach of Islam et al. [24] could be a valuable next step.

Additionally, this project primarily focused on supervised learning. Future research could explore the potential of semi-supervised and unsupervised learning approaches, as demonstrated by Lee et al. [23], to enhance the robustness and applicability of intermittent SNNs.

Lastly, this project was developed using SNNTorch without any neuromorphic hardware. To validate the results presented in this report, it is necessary to implement the developed EMG SNN on neuromorphic hardware and verify the calculated inference overheads are accurate.

Chapter 7

Conclusion

The overall objectives of this project were to assess the possibility of making an SNN intermittent, define its trade-offs and then evaluate its effect on an SNN EMG model. Our results show that our simulations and data show that it is possible to create an intermittent SNN. Furthermore, we found the trade-off to be primarily a medium increase in energy consumption and an incredibly large increase in latency. Lastly, we found that altering the architecture of an SNN (i.e. creating a different model) results in different checkpointing overhead, frequency and location.

Overall, due to time limitations, this project only scratched the surface of intermittent SNNs, and further work is needed to both confirm this project's results and better understand the general dynamics between ImC and SNNs.

References

[1] Lara McManus et al. "Consensus for experimental design in electromyography (CEDE) project: Terminology matrix". In: *Journal of Electromyography and Kinesiology* 59 (2021), p. 102565.

[2] John Hopkins Medicine. *Electromyography (EMG)*. Accessed: 2024-07-30. URL: https://www.hopkinsmedicine.org/health/treatment-tests-and-therapies/electromyography-emg.

[3] Charu C. Aggarwal. "An Introduction to Neural Networks". In: *Neural Networks and Deep Learning: A Textbook*. Cham: Springer International Publishing, 2018, pp. 1–52. ISBN: 978-3-319-94463-0. DOI: 10.1007/978-3-319-94463-0_1. URL: https://doi.org/10.1007/978-3-319-94463-0_1.

[4] Frank Rosenblatt. "The perceptron: a probabilistic model for information storage and organization in the brain." In: *Psychological review* 65 6 (1958), pp. 386–408. URL: https://api.semanticscholar.org/CorpusID:12781225.

[5] Stefan Leijnen and Fjodor van Veen. "The Neural Network Zoo". In: *Proceedings* 47.1 (2020). ISSN: 2504-3900. DOI: 10.3390/proceedings2020047009. URL: https://www.mdpi.com/2504-3900/47/1/9.

[6] Catherine D. Schuman et al. "Opportunities for Neuromorphic Computing Algorithms and Applications". English. In: *Nature Computational Science* 2 (2022), pp. 10–19. ISSN: 2662-8457. DOI: 10.1038/s43588-021-00184-y. URL: https://www.nature.com/articles/s43588-021-00184-y.

[7] Jason K. Eshraghian et al. "Training Spiking Neural Networks Using Lessons from Deep Learning". In: *Proceedings of the IEEE* 111.9 (2023), pp. 1016–1054. ISSN: 15582256 and 00189219. DOI: 10.1109/JPROC.2023.3308088.

[8] Dezhen Xiong et al. "Deep Learning for EMG-based Human-Machine Interaction: A Review". In: *IEEE/CAA Journal of Automatica Sinica* 8.3 (2021), pp. 512–533. DOI: 10.1109/JAS.2021.1003865.

[9] Zhen Ding et al. "sEMG-Based Gesture Recognition with Convolution Neural Networks". In: *Sustainability* 10.6 (2018). ISSN: 2071-1050. DOI: 10.3390/su10061865. URL: https://www.mdpi.com/2071-1050/10/6/1865.

[10] Yu Hu et al. "A novel attention-based hybrid CNN-RNN architecture for sEMG-based gesture recognition". In: *Pattern Recognition* 122 (2022), p. 108228.

[11] Manfredo Atzori, Matteo Cognolato, and Henning Müller. "Deep Learning with Convolutional Neural Networks Applied to Electromyography Data: A Resource for the Classification of Movements for Prosthetic Hands". In: *Frontiers in Neurorobotics* 10 (2016), p. 9.

[12] Simon Tam et al. "A Fully Embedded Adaptive Real-Time Hand Gesture Classifier Leveraging HD-sEMG and Deep Learning". In: *IEEE Transactions on Biomedical Circuits and Systems* 14.2 (2020), pp. 232–243. DOI: 10.1109/TBCAS.2019.2955641.

[13] Marcello Zanghieri et al. “Robust Real-Time Embedded EMG Recognition Framework Using Temporal Convolutional Networks on a Multicore IoT Processor”. In: *IEEE Transactions on Biomedical Circuits and Systems* 14.2 (2020), pp. 244–256. DOI: 10.1109/TBCAS.2019.2959160.

[14] Antonio Vitale et al. “Neuromorphic Edge Computing for Biomedical Applications: Gesture Classification Using EMG Signals”. In: *IEEE Sensors Journal* 22.20 (2022), pp. 19490–19499. ISSN: 15581748 and 1530437x. DOI: 10.1109/JSEN.2022.3194678.

[15] Sai Sukruth Bezugam, Ahmed Shaban, and Manan Suri. “Low Power Neuromorphic EMG Gesture Classification”. English. In: (2022). Available at: https://arxiv.org/abs/2206.02061. DOI: 10.48550/arXiv.2206.02061.

[16] Enea Ceolini et al. “Hand-Gesture Recognition Based on EMG and Event-Based Camera Sensor Fusion: A Benchmark in Neuromorphic Computing”. In: *Frontiers in Neuroscience* 14 (2020), p. 637. ISSN: 1662453x and 16624548. DOI: 10.3389/fnins.2020.00637.

[17] Saad Ahmed et al. “The Internet of Batteryless Things”. In: *Communications of the ACM* 67.3 (2024), pp. 64–73. ISSN: 15577317 and 00010782. DOI: 10.1145/3624718.

[18] Sumanth Umesh and Sparsh Mittal. “A survey of techniques for intermittent computing”. In: *Journal of Systems Architecture* 112 (2021), p. 101859. ISSN: 1873-6165, 1383-7621. DOI: 10.1016/j.sysarc.2020.101859.

[19] Brandon Lucia et al. “Intermittent Computing: Challenges and Opportunities”. In: *Leibniz International Proceedings in Informatics, Lipics*. Vol. 71. 2nd Summit on Advances in Programming Languages, SNAPL 2017. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017, p. 8. ISBN: 3959770324 and 9783959770323. DOI: 10.4230/LIPIcs.SNAPL.2017.8.

[20] Hrishikesh Jayakumar, Arnab Raha, and Vijay Raghunathan. “QUICKRECALL: A Low Overhead HW/SW Approach for Enabling Computations Across Power Cycles in Transiently Powered Computers”. In: *Proceedings of the IEEE International Conference on VLSI Design*. 27th International Conference on VLSI Design, VLSID 2014 - Held Concurrently with 13th International Conference on Embedded Systems Design. IEEE, 2014, pp. 330–335. ISBN: 1479925128, 1479925136, 1479925144, 9781479925124, 9781479925131, 9781479925148. DOI: 10.1109/VLSID.2014.63.

[21] Kiwan Maeng and Brandon Lucia. “Adaptive Dynamic Checkpointing for Safe Efficient Intermittent Computing”. In: *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2018, pp. 129–144. ISBN: 1939133084, 9781939133083.

[22] Alexei Colin and Brandon Lucia. “Termination Checking and Task Decomposition for Task-Based Intermittent Programs”. In: *CC 2018 - Proceedings of the 27th International Conference on Compiler Construction, Co-located with CGO 2018*. Association for Computing Machinery, Inc, 2018, pp. 116–127. ISBN: 1450356443, 9781450356442. DOI: 10.1145/3178372.3179525.

[23] Seulki Lee et al. “Intermittent Learning: On-Device Machine Learning on Intermittently Powered Systems”. In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 3.4 (2019), p. 141. ISSN: 24749567. DOI: 10.1145/3369837.

[24] Bashima Islam and Shahriar Nirjon. “Zygarde: Time-Sensitive On-Device Deep Inference and Adaptation on Intermittently-Powered Systems”. In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4.3 (2020), p. 82. ISSN: 24749567. DOI: 10.1145/3411808.

[25] Graham Gobieski, Nathan Beckmann, and Brandon Lucia. “Intermittent Deep Neural Network Inference”. In: *SysML*. 2018.

[26] Udayanga K. N. G. W. Gamage et al. “Event-Based Classification of Defects in Civil Infrastructures with Artificial and Spiking Neural Networks”. In: *Lecture Notes in Computer Science*. Vol. 17. Springer, 2023, pp. 629–640.

[27] Edgar Lemaire et al. “An Analytical Estimation of Spiking Neural Networks Energy Efficiency”. In: *29th International Conference on Neural Information Processing, ICONIP 2022*. Vol. 13623. Lecture Notes in Computer Science. Virtual Event: Springer, 2023, pp. 574–587. ISBN: 9783031301056. DOI: 10.1007/978-3-031-30105-6_48.

[28] Manfredo Atzori et al. “Electromyography data for non-invasive naturally-controlled robotic hand prostheses”. In: *Scientific Data* 1.1 (2014), p. 140053. DOI: 10.1038/sdata.2014.53.

[29] Catherine D. Schuman et al. “Non-traditional input encoding schemes for spiking neuromorphic systems”. In: *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–10. DOI: 10.1109/IJCNN.2019.8852139.

[30] Daniel Auge et al. “A Survey of Encoding Techniques for Signal Processing in Spiking Neural Networks”. In: *Neural Processing Letters* 53.6 (2021), pp. 4693–4710. DOI: 10.1007/s11063-021-10562-2.

[31] Fujitsu Semiconductor. *MB85RC256V 256 Kbit (32K × 8) I²C FRAM*. Accessed: 2024-07-14. 2017. URL: https://www.fujitsu.com/uk/Images/MB85RC256V-20171207.pdf.

[32] Ivaylo Pandiev et al. “Analysis of the Methods for Realization of Low-Power Piezoelectric Energy Harvesting Circuits for Wearable Battery-Free Power Supply Devices”. In: *Applied Sciences* 14.11 (2024), p. 4792. DOI: 10.3390/app14114792. URL: https://doi.org/10.3390/app14114792.

[33] F.R. Pathan. “Kinetic Energy Harvesting from Human Hand Movement by Mounting micro Electromagnetic Generator”. In: *E3S Web of Conferences*. Vol. 115. Published online: 02 September 2019. The 2nd International Conference on Electrical Engineering and Green Energy (CEEGE 2019). 2019, p. 02005. DOI: 10.1051/e3sconf/201911502005. URL: https://doi.org/10.1051/e3sconf/201911502005.

Appendix A

Inference Energy Calculations

Layer 1 parameters:

- $\theta_{l-l} = 348$
- $C_{out} = 64$
- $W_{kernel} = 1$
- $H_{kernel} = 60$
- $H_{kernel} = 60$
- $W_{out} = 12$
- $H_{out} = 1$
- $N_{output} = 4$
- $T = 10$
- $S = 1$
- $\theta = 8068$

Energy Consumption memory access (E_{mem}):

$E_{mem} = (RdIn + RdParam + RdPot) \times E_{RdRAM} + (WrOut + WrPot) \times E_{WrRAM}$

$RdIn = \theta_{l-l}$ $RdIn = 348$

$RdParam = \theta_{l-l} \times C_{out} \times W_{kernel} \times H_{kernel} + C_{out} \times W_{out} \times H_{out}$

$RdParam = 348 \times 64 \times 1 \times 60 + 64 \times 12 \times 1$

$RdParam = 1337088$

$RdPot = \theta_{l-l} \times C_{out} \times W_{kernel} \times H_{kernel} + C_{out} \times H_{out} \times W_{out}$

$RdPot = 348 \times 64 \times 1 \times 60 + 64 \times 1 \times 12$

$RdPot = 1337088$

$E_{RdRAM} = 100 * 10^{-}12$ [Lemaire et al.]

$WrOut = N_{output}$ $WrOut = 90$

$WrPot = \theta_{l-l} \times C_{out} \times W_{kernel} \times H_{kernel} + C_{out} \times H_{out} \times W_{out}$

$WrPot = 348 \times 64 \times 1 \times 60 + 64 \times 1 \times 12$

$WrPot = 1337088$

$E_{WrRAM} = E_{RdRam}$

$E_{mem} = (348 + 1337088 + 1337088) \times 100 * 10^{-}12 + (90 + 1337088) \times 100 * 10^{-}12$

$E_{mem} = 0.0004011702(J)$

$E_{mem} = 401.17(uJ)$

Energy Consumption memory access ($E_{ops+addr}$):

$E_{ops+addr} = (E_{Add} + E_{Mul}) \times MAC_{Ops+Addr} + E_{Add} \times ACC_{Ops+Addr}$

$E_{Add} = 0.1 * 10^{-}12$ [Lemaire]

$E_{Mul} = 3.1 * 10^{-}12$ [Lemaire]

$MAC_{Addr} = \theta_{l-l} \times 2$ $MAC_{Addr} = 348 \times 2$ $MAC_{Addr} = 696$

$MAC_{Ops} = T \times C_{out} \times H_{out} \times W_{out}$ $MAC_{Ops} = 10 \times 64 \times 1 \times 12$ $MAC_{Ops} = 7680$

$ACC_{ops} = \theta_{l-l} \times (H_{kernel}/S) \times (W_{kernel}/S) \times C_{out} + T \times C_{out} \times H_{out} \times W_{out} + \theta_l$

$ACC_{ops} = 348 \times (60/1) \times (1/1) \times 64 + 10 \times 64 \times 1 \times 12 + 8068$

$ACC_{ops} = 1352068$

$ACC_{Adr} = \theta_{l-l} \times C_{out} \times H_{kernel} \times W_{kernel}$ $ACC_{Adr} = 348 \times 64 \times 60 \times 1$

$ACC_{Adr} = 1336320$

$E_{ops+addr} = (0.1 * 10^{-}12 + 3.1 * 10^{-}12) \times (7680 + 696) + 0.1 * 10^{-}12 \times (1352068 + 1336320)$
$E_{ops+addr} = 0.000000295642(J)$ $E_{ops+addr} = 0.295642(uJ)$

$E = E_{mem} + E_{ops+Addr}$ $E = 401.17 + 0.295642$ $E = 401.47uJ$

Technical
University of
Denmark

Brovej, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk